

Project 1: Custom RTOS

April 24, 2026 3:26 AM

So I am using my ARM Cortex MCU for this custom RTOS implementation

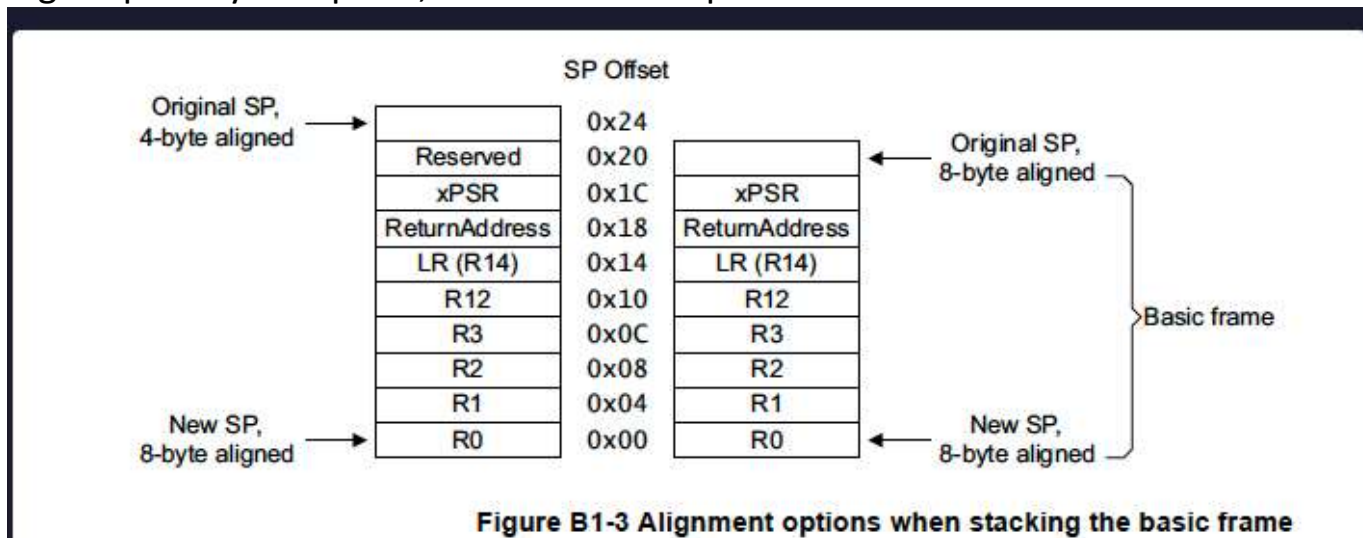
Milestones:

1. Preemptive round-robin with 3 tasks running
2. Full kernel with sync primitives, heap, and sleep/timer
3. Live Gantt chart visualizer running, GitHub + Live demo

Notes from Milestone 1 - Context Switching and how it works in ARM Cortex-M

- Main Stack Pointer (MSP): Default stack used for OS kernels, exception handlers, and privilege code.
- Process Stack Pointer (PSP): Used for applications threads to prevent them from corrupting the system stack.
- Handler Mode: Strictly for handling exceptions and interrupts (systick, pendSV, and external interrupts). **Always Privileged**
- Thread Mode: Normal Application Code -> RTOS tasks -> configured to be privileged or unprivileged.
- Veneer: it is a small, linker-generated piece of code that bridges function calls, primarily used to handle long-distance branches or state changes. R12 is the only ARM cortex register that can be used within a veneer without needing to preserve the original state.
- R9 can be used as a thread register, which holds the pointer to the current thread-local storage context
- If unprivileged code attempts to read any stack pointer, the priority masks, or the IPSR, the read returns zero.
- Privileged: Has full access to all memory regions. It can read and write to all critical system registers, including the NVIC and the SysTick timer.
- Unprivileged: Cannot access system control registers (NVIC, SysTick, SCB). If an unprivileged task tries to disable an interrupt or write to the SysTick reload register, the CPU will instantly throw a HardFault
- Cannot change the execution state back to privileged directly.
- Has restricted access to certain special registers (like CONTROL)

- The CONTROL register has the last 2 bits which determine how Thread mode behaves
- Bit [0] -> **nPRIV**: 0:Privileged, 1:Unprivileged
- Bit [1] -> **SPSEL**: Stack Pointer Selection: 0: Uses the main stack pointer, 1: Process Stack Pointer
- Bit [2] -> **FPCA**: indicates whether the floating point context is active.
- Bit [3] -> **SFPA**: Indicates whether secure floating-point is active or inactive based on whether the bit is set or not. This is only active if the Security Extension is implemented.
- We can only write to the CONTROL register if we are currently in a privileged state.
- PendSV: Designed for context-switching
- Note: if the system was in thread mode using the PSP and exception takes place, the data is pushed onto the PSP. If the core was already servicing another exception and was preempted by a higher priority exception, the data will be pushed onto the MSP.



EXC_RETURN Value	Mode to Return To	Stack to use
0xFFFFFFF1	Handler Mode	MSP
0xFFFFFFF9	Thread Mode	MSP
0xFFFFFDFD	Thread Mode	PSP
0xFFFFFEE1	Handler Mode (FPU Extended Frame)	MSP
0xFFFFFEE9	Thread Mode (FPU Extended Frame)	MSP
0xFFFFFEDD	Thread Mode (FPU Extended Frame)	PSP

RTOS Context Switching:

- Scheduler: Capable of **context switching** between different tasks.
Can be **Preemptive** -> A context switch while a task is running if something more important comes up
Cooperative -> A context switch NEVER occurs while another task is running.
- Masking an interrupt: Temporarily ignoring or disabling it

A context switch using pendSV happens in six phases:

1. An exception (pendSV) triggered -> CPU pushes 8 critical registers (R0-R3, R12, LR, PC, xPSR) to the PSP.
2. It looks at the current task's stack pointer and manually saves the remaining core registers (R4-R11) onto that stack. Even the FPU is saved. At this point, the task's brain is essentially frozen on stack
3. The code takes the final updated memory address of that stack and saves it into the old task's TCB. This ensures that the OS knows where to look.
4. For freeRTOS, we use the vTaskSwitchContext which is the actual scheduler. It looks at priorities, states, and decides which tasks to run next (we can implement this in simple C). Then, it updates a global pointer to point to the new task's TCB.

5. The code reads the new task's TCB to find where its stack was left off. It then does the reverse of number 2, manually pops the saved R4-R11 and any FPU state off the new task's stack and back into the CPU registers.
6. Code points to the hardware PSP register to the newly restored stack and triggers an exception return. The ARM hardware takes over again and automatically pops off those 8 critical registers off the stack.

A common pattern that the RTOS follows is -> it initialises the task stack when creating a new task to look like it had been context switched out by the scheduler. Then it starts the scheduler itself by triggering an SCV exception instruction. This way, **starting a thread** is identical to context switching to a thread.

- When zeroing out an array of structs, (In our case we are working with the task pointer of the struct in an array of structs) run a for loop, and look inside the struct and zero that. Just equaling it to {0} wouldn't work, and ensure u know the types as well
- We first work with the OS. It needs to have a Task Control Block (TCB) which stores the TASK at hand. It stores the stack pointer, and an array for the task to store its stuff in. this array acts like RAM for that specific task.
- We keep track of the current task using a pointer
- and we also make an array of tcb structs. It stores all the tasks.

- Then, in the core of the OS, we initialise the OS by zeroing out the stack pointer of each task's TCB
- CreateTask -> this gets a bit complicated. For it to work, it needs to know which slot does the task go into from the task_array. It also takes in WHAT C code it must run, which comes from a pointer.
- ARM Stack grows downwards -> so we start from the highest address and work downwards. So, the starting stack pointer should point to the last element.
- When PendSV fires and does an exception return, the CPU hardware automatically pops 8 registers off the PSP stack in the order of
PSR, PC, LR, R12, R3, R2, R1

- Remember that stack grows downward, so you decrement your stack pointer first, then write. (pre-decrement basically)
- We have 16 words in total (16 registers).
- 8 registers are automatically saved on any exception entry (silicon)
- 8 Registers (R4-R11) that your handler will save manually by convention.
- So out of all the various values of EXC_RETURN, we choose 0xFFFFFDD. This ensures that the exception return uses non-floating point state from the PSP and the execution uses PSP after return. Mainly, this allows us to return to thread mode.
- This basically allows us to create a 'return ticket' (gemini, 2026). When we trigger the first PendSV, the CPU reads D and switches to Thread mode, grabs the carved out 8 register values and pops the dummy task.
- So this was basically our startup - we have solved the issue where even the first task functions like a switch. Refer to YELLOW highlighted part above to see what it does in terms of RTOS functionality.
- CONTROL Register is used to control the stack used and the privilege level when the processor is in thread mode and indicates whether the FPU state is active or not (TRM)
- __ISB() (Instruction Synchronization Barrier) in ARM is used to flush the CPU Pipeline, ensuring that all instructions following it are fetched and executed only after previous context-altering instructions like the MPU, cache, or system register changes have been completed.
- It guarantees strict order for instruction fetching and context.
- We used ISB after changing bit 1 of the CONTROL Register. We use it to prevent pipeline incoherence. Without it, the processor may attempt to use the old stack pointer for instructions that were already fetched before the change was realised by the hardware.
- SVCall - Supervisor Call (SVC) is an exception that is triggered by the SVC instruction. In an OS environment, applications can use SVC instructions to access OS kernel functions and device drivers

- Hard Fault: Triggered when a fault occurs that cannot be managed by any other mechanism. Example: Handler Failure (fault occurring within a fault handler itself).
 - Memory Management Fault: when a memory access is blocked by the MPU (memory protection unit) or fixed memory constraints. Example: Access Violations (attempting to write to a region marked as read-only or accessing privileged data from unprivileged code)
 - Bus Fault: when the bus system returns an error response during an instruction fetch, data read, or data write. Example: Hardware issue, device errors, invalid addresses
 - Usage Fault: various execution errors, some of which are configurable. Example: Undefined instructions.
-
- LR is the Link Register. Normally LR holds a return address — where to go back to after a function call. But in exception handlers, ARM repurposes LR to hold a special value called **EXC_RETURN**.
 - Specifically 0xFFFFFDD means: **return to thread mode, use PSP, no FPU**.
-
- PendSV is an exception that exists specifically for context switching. It has one job in our OS. SysTick fires periodically and says it is time to switch. But, systick doesn't do the switch itself. It just pends pendSV. PendSV then fires at the lowest priority, after every other interrupt has been finished, and does the actual swap.
 - Mental Mode: We have two tasks. Each has its own PSP pointing into its own tcb_array. At any moment, one task is running and one is frozen. When PendSV fires, your job is to **Freeze the current task (save everything it was doing onto its own stack), Decide who runs next (call your C scheduler), and Thaw the next task (restore everything from its stack)**
-
- Thumb Instructions - This was made by ARM engineers to save space. They took the most common instructions and compressed them down to fit into exactly 16 bits (2 bytes).
 - STRELOAD Register: This sets how many clock cycles are between each tick. We put in 11999 as the value (read up the TRM) as our clock speed is 120MHz and for 1ms delay, we must go through

120,000 cycles.

- STCURRENT Register: Write any value to this to clear it and reset the counter to zero before you start
- STCTRL: This is the control register. Three bits matter - counter enable, SysTick interrupt enable, and clock source selection. This enables the SysTick features.
- Extern is used to tell the compiler that this function exists in ANOTHER file. So, we use it on a declaration in the file we need to bring it INTO. Same thing with variables - it guarantees that every single file in your project is looking at the exact same physical chunk of RAM. Without Extern, the compiler will make brand new, separate variables in every .c files they exist in.
- In a true preemptive RTOS, the tasks are completely ignorant of the system clock.
- SysTick is the hardware running in the background in this case of OS. Every 1ms, the systick fires and runs the pendsv context switcher, and hands the CPU to the next task
- So we cant call systick normally for delays. We use the OS function, OS_Sleep() which tells the CPU to wait for 500ms as we don't need to run anything here (basically go to sleep for 500ms)

(MILESTONE 1 DONE - I was able to flash all 4 LEDs at the same time - they all flashed at different delays. So, 4 different tasks were running at the same time.)

- Our Task must be in either READY, RUNNING, or BLOCKED states. We can use an ENUM for this in the TCB struct.
- The scheduler should skip the BLOCKED and RUNNING ones, and pick the next READY one. Tasks start as READY.
- Now, I added the priority based scheduling. The struct has a new uint8_t priority field. Also, the createTask function has a priority parameter.
- Then, I used it within the OS_Schedule function. We NEED to make sure that the next task being picked from the task_array is READY (in terms of state) and has the highest priority of all the tasks in the

array.

- This was done by using a simple loop through the tasks, check their states AND priorities - if they met the criteria, we set a new highest priority value, and set an index tracker to that index. Then, when the loop ends, we ensure that the currenttask now points to the task_array's new index which has the highest priority and state=READY.
- Now, we are working with semaphores and mutexes -we know how it works in theory - but, the kernel level depth will be enhanced using the definitions below.
- **Binary Semaphore** at the kernel level - It is a counter that's either 0 or 1. the two operations are
 1. **Wait** (also called **acquire** or **P**) - If the counter is 1, decrement it to 0 and let the task continue. If it is 0, the task can't proceed - It gets marked BLOCKED and the scheduler picks someone else.
 2. **Signal** (also called **release** or **V**) - Increment the counter back to 1, and wake up any task that was BLOCKED waiting on it.
- The key insight is that a semaphore has NO OWNER. Use case: Task A does some work, signals the semaphore, task B was waiting and now wakes up.
- **Mutex** at the kernel level - either a 0 or 1. BUT, it HAS OWNERSHIP. The task that locks it is the only task allowed to unlock it. IT NEEDS TO TRACK whether it is locked or not and which task currently owns it.
- SOOO, in terms of actual C code, we created the semaphore struct using a counter variable (0 or 1), an array of pointers to TCBs (tasks) which are waiting on it, and a count variable which counts the number of tasks waiting (how many elements in that array)
- For Mutex, we keep the same structure, but add the owner field - it will be a pointer to a TCB (task) which shows the owner of this key (mutex) and is the only one that can lock/unlock it.
- So we know what priority inversion and inheritance are from before in broad terms. In a more streamlined definition, Priority Inversion is: Task A has a low priority and holds a mutex. Task B cant run until task A releases it. But task A keeps getting preempted by medium-priority tasks, so it never gets CPU time to finish and

release the Mutex. Task B is stuck forever even though it is high priority. (remember the stapler and the CEO and the intern example)

- Priority Inheritance: When a high priority task blocks on a mutex owned by a low priority task, the owner temporarily inherits the higher priority so it can finish and release the mutex quickly.
- Something I was really wondering was the use cases of mutex and semaphores.
- Mutex: Shared Hardware - Imagine two tasks both wanting to print debug over UART. UART is one peripheral. If both tasks write to it at the same time, the bytes misbehave and we get garbage results over uart. With a Mutex, it will protect the UART by allowing a task to lock it.
- Semaphore: It is a producer/consumer pattern. Task A reads sensor data and puts it in a buffer. Task B needs to process this data and will not run until task A puts the data in a buffer. A semaphore starts at 0. Task B calls Wait() and immediately blocks. When task A finishes filling up the buffer, it calls Signal() and the semaphore goes to 1. Task B wakes up and processes.
- Implementation of OS_Signal() -> Increment the counter to 1; check if any tasks are currently waiting on it; search for the highest priority task - Grab the pointer at that index, change its state to READY; Run another loop until TaskTracker-1; tell the array to copy the person sitting in the next chair (i+1) to the current chair (i).; Once the loop finishes, the hole has been successfully pushed to the end of the line. Overwrite that with NULL and decrement TaskTracker.
- Ok so I got a bit confused while doing the Mutex implementation. Mainly at the difference between CurrentTask and owner of the mutex. Basically, let's say Task A (priority 1) calls Lock() and gets the mutex. It is running. SysTick fires, Task B (pr 3) gets scheduled. Task B also calls Lock(). The mutex however is already taken. In conclusion - Mx->owner is whoever holds the mutex. They got it earlier and haven't released it yet. CurrentTask is the task whose code is executing it right now. It is whoever called Lock() at this

moment.

- Despite having written notes in the 'iPad Designing' File, we will do some explanation as to how I developed the Mutex functions in detail:
- `Mutex_Lock()` -> Checks if the counter is 1 - if yes, it sets the current task as the owner of the mutex (currenttask now holds the mutex), saves the original priority, and decrements the counter to 0 (shows that the mutex is taken). If not, we know that the mutex is taken by some other task - this means, we go into the else block of the function. So, we compare this new CurrentTask's (trying to access the mutex) priority with the owner's priority. If the current task has a higher priority, we need to elevate the owner's priority to this priority so that the owner completes its task first and then is able to hand over the mutex to this currentTask. This is priority inheritance. Then, we block the current task, add it to the waiting task list, decrement the tracker, and do the context switch to the next task.
- Preemption: CPU moves to a higher priority task.
- Understanding the Heap Allocator: First, let's figure out what it really is - it is like a manager. Right now, all the memory is stored statically at compile time. The heap allows tasks to dynamically ask for memory while the system is running - so stuff like `malloc()` and `free()`. We want to include this in our OS for RTOS. For this, we need to create a huge array. However, we need to know which parts of this array are taken and which are free - this can be done using a metadata header. This is a header struct - this header is like a binding document of memory transfer. It needs size and a boolean indicating free/not free. We also need stack overflow detection - for this, just use the first index of the `tcb_array` (RAM for each task) which will have a canary value (stack grows downward). If this is ever overwritten, we know that the stack has overflowed. We can use the industry standard value, `0xDEADBEEF` as the canary.
- Let's design the allocator and overflow detection (Rough (ipad dead lmao)):

So, we have the malloc.c file. We need to start by initialising my Block Header Struct. This is essentially a piece which tells us the size and availability in the heap array. So, this will include a field called size, probably uint32_t and shows the number of BLOCKS essentially let's say 32 bit blocks, wait that makes it 1.... So we just set it to 1 for now I guess. and another bool field called is_free, 0 (not free), 1 (free).

Then, we can get to the os_malloc(size) function. Essentially, the first thing I would do is work with the header size. Multiply that size field by the function parameter and that's the number of blocks we need to carve. If size=3, we need to carve out 3 blocks, $32 \times 3 = 96$ bits = 12 Bytes of data in the heap RAM. BUT FIRST, let's check if the blocks, starting from the first one, are free - keep moving up the heap until a free block is hit. Once that's done, THEN do the size part - then, change is_free to 0 cuz it won't be free anymore, we're gonna allocate it for this thing. Also, the free block needs to be BIG enough, so 12 bytes, so in this case, we're saying like three 32 bit integers.

Os_free() function will essentially mark that specific BLOCK as Free. For the Canary, we can shift our focus to the OS_CORE stuff. When create_Task happens, we can essentially write 0xDEADBEEF at index 0 of tcb_array[]. We have a universal function called OS_Fault now, so SysTick_Handler needs to check every task's canary on each tick - as soon as it gets overwritten, we call OS_Fault which blinks LED1 and also indicates there is stack overflow.

OK SO CLAUDE REFINEMENTS FOR THIS:

For the size - think in Bytes, not Bits. Let's say size PARAMETER for the function is 12. We need to keep looping through the heap until we find size>=12.

We start at the beginning of Heap. Read the Header. If it is FREE and BIG ENOUGH (size>=parameter_size), we use it. Else, we NEED to skip forward by header size + block size.

If we find a block of 100 bytes, and we only need 12, we don't mark all of them as used. We carve out the 12-byte block WITH ITS OWN header and then write a new header for the remaining 88 bytes to mark them as free.

Think about OS_Free() - I said we can mark the Block as free.

However, `OS_Free()` will only get a pointer to the data needed to be freed. Think about how to get from pointer to header.

The Canary stuff is good to go - just implement it after working on the `malloc()` functions and files.

Also, remember that `os_malloc()` will return a generic pointer to the first byte of that allocated space.

Whatever `malloc` returns, goes EXACTLY into the input parameter of `os_free`.

- The Header can be explained as a mailbox of a house which says 'this house is 50m wide and currently empty'. The house is where the data lives. In memory, it looks like:

[Header][...data....][Header][...data....][Header][...data....] - Every allocation has exactly one header immediately before it. The header tells you how big the data region following it is, and whether it is free.

- So ive been a little confused with the pointer math here so let's clear that up. Let's say the Current Header pointer is pointing to memory address 100. We know that the header is 8 bytes long from `sizeof(header)`. We know that the payload is 32 bytes long from `current->size`. We are looking at this in RAM:

[Header of 8 bytes][Payload of 32 bytes]

This means if we did not find the desirable block, we jump forward by $8 + 32 = 40$ steps in memory. Therefore, the next header must physically start at an address of 140.

Now when it comes to actually returning a pointer to the payload, we need to be careful with the math. Also, I was way too focused on arrays whereas this is just a simple thing. If u have a header of 8 bytes and right after u have a payload, so how much does my current header pointer have to jump to get to the payload???

Simple, 8 bytes. And right now, our current pointer points to something which IS 8 bytes. So, we say `Current + 1` gets me to the payload. We return this. In C, adding 1 to a pointer always adds $1 * \text{sizeof}(\text{the_thing_it_points_to})$. Also, don't confuse with normal pointer which in our architecture is 4 bytes long.

- Now, let's discuss how we work with empty space. If let's say the size requested is 32 bytes and we have a 100 byte payload - so we

would obviously not want to waste the remaining space. To fix this, we use similar pointer math to create a new header pointer called leftoverBlock. We cast current to a raw byte pointer - Add the size of the header - Add the exact size the task just requested - cast the new calculated address into a brand new pointer leftoverBlock.

Now, set its is_free to 1 and find the new size using original block size - requested size. - the 8 bytes we just burned for the header.

- There is something called the Fragmentation Threshold - If a block is 40 bytes and the task is 32 bytes, it leaves 8 bytes. However, our header is also 8 bytes so there is no space to put both a header and payload from this. SOOO, we leave it. You just give the task the ENTIRE block.
- For the free part, we take in the pointer to the start of the payload which we allocated in memory for the task. So, we need to set its block to free. We step back by header length, whatever that points to is the header for the payload which was allocated. We mark it as free and we're done.
- I designed a OS_Sleep(ticks) function which made the task sleep for N ticks. It allowed a task to sleep without using up CPU cycles as the CPU goes onto spend time on other tasks using a context switch (PendSV set pending). It goes through all tasks. Checks if the task is BLOCKED already and has a number of ticks (a new field in the TCB struct) greater than 0. If true, we decrement the ticks and context switch to another task. Within the same if statement, we check if the counter (ticks) reaches 0. If true, we set the task's state to READY which wakes it up.
- Now, we are onto the ITM (Invisible console). It stands for Instrumentation Trace Macrocell. Refer to this before designing anything - file:///C:/Users/zainn/Downloads/coresight_v3_0_architecture_specification_IHI0029E.pdf
- Now, we can read up on the general descriptions and then move on to the TRM chapter 4 which explains this really well as well.
- Normally, if we want to use print debug messages to your PC, we

use UART or printf messages but if we use this, the task has to wait around for bits to slowly trickle out of a pin. This delay might cause some high-priority tasks to miss their real-time deadlines. ITM is a piece of hardware directly built into my ARM processor which has an ultra-fast data pipeline designed specifically for debugging.

When you write data to the ITM, the CPU spends exactly one clock cycle dropping the data off, and then immediately goes back to running your RTOS. The ITM hardware handles it completely in the background.

- The Pipeline:

There is a pin on our MCU called the SWO (Serial Wire Output) pin. Here is a step by step journey of a single character 'A' being transported:

1. The Code (Stimulus Ports): The ITM has 32 memory-mapped registers called stimulus ports. Our code writes the letter 'A' into stimulus port 0 (default port for text)
2. ITM Hardware: Silicon detects the letter. Attaches a tiny hardware timestamp to it, and packages it into a trace packet.
3. The TPIU (Trace Port Interface Unit): The ITM hands the packet to the TPIU, which is the hardware bouncer. The TPIU serializes the packet and blasts it out of the physical SWO pin on our board at very high speeds.
4. Debug probe: our CMSIS DAP debugger is connected to the SWO pin.
5. Keil receives the USB data and unpacks the ITM to print the letter 'A' in the viewer window.

- Based on the architecture specifications, we are using Software Lock Status Registers (LSR) and Software Lock Access Registers (LAR). This lock mechanism prevents accidental access to the registers of the CoreSight components.

- Writing 0xC5ACCE55 signals the LSR that it must permit writing to other registers in the component.

- LAR = 0xFB0

- LSR = 0xFB4

- One clarification - the ITM doesn't always take only ONE cpu cycle. If the buffer is full, the CPU will stall for a bit then move on. This

shouldn't be an issue in our implementation as our context switches and semaphores are not as frequent.

- Also, during initialisation, the first step is to use the Debug Exception and Monitor Control Register (DEMCR). It needs its trace enable bit set. Specifically, bit 24, called TRCENA. Without this setting, the entire subsystem is turned off and does not work.
- Reference used for ITM: https://web.eecs.umich.edu/~prabal/teaching/eecs373-f10/readings/ARMv7-M_ARM.pdf
- Just created an idle task after testing the LEDs with `OS_Sleep()`. It is a `while(1)` forever loop and has a priority of zero. It is created in `Init_OS` and stored at the last index of `task_array`.
- Now, I'm trying to make an ITM packet. Considering we have 32 bits in total, the top 8 bits are the event (malloc, signal, wait, etc.), the next 8 bits are the task index, and the bottom 16 bits are the ticks. We made a macro which turned all three things into a 32-bit number. Then, shifted the event 24 to the left, making it the first 8 bits of the packet. Then, shifted the index 16 bits to the left to make it the next 8. Then, just left the ticks as it is, making it the last 16 bits in the packet. These were all OR'ed together to create the packet. This is similar to the way vehicle telemetry data works and how we piece stuff together using bitwise operation.
- Some notes on the ITM viewer: the ITM hardware wraps my data in a hardware header. It contains a port ID and a timestamp. The port ID is which of the 32 stimulus ports did this data come from. The timestamp is the CPU cycle at which this happened (`OS_Ticks` for us).
- The TPIU hardware blasts this packaged data out of the SWO pin, our debug probe catches it, and sends it over USB to PC.
- Deep inside the debugging engine is a background service listening to that USB data stream. When a packet arrives, Keil looks at the port ID, and uses that ID to decide which window to display it on.
- Stimulus port 0 is human readable text. Any packet tagged with port 0 gets sent into printf window. It assumes the payload is standard ASCII
- Ports 1-32 - Left as Binary (Custom OS Trace packets, logic analyzer,

etc.)

- So, in our case, let's send the packet to port 1 as a single binary value. Python will read these raw bytes and decode them. SOOO, port 0 is only used for basic messages, like printf, such as 'task started', etc. for the gantt chart, we would need structured binary data.
- Let's use Keil's .log file which receives the data from the ITM while the target runs. Python reads this file.
- Remember how we worked on tracing out vehicle telemetry for the geotab python interview??? We use the same approach here. The packet is split into 8,8,16 bits and makes up 4 bytes of data. This means that we create masks to split the 32-bit binary number into these groups, read them separately, and build a timeline to give to matplotlib.
- Final Algo: Soo, we were overcomplicating it. We just need to use simple masks, and not loops (ok I was sleepy while writing the loop, I see now it doesn't work lowkey). We just pull out fields using masking. Three lines, no loops. So, first thing is open the log file, then create an empty dictionary. THEN, run a loop which reads 4 bytes at a time only until end of file. Unpack, extract, see if event is the same as TASK_SWITCH and if so, check if current task actually exists. If so, close previous slice - add current_start, timestamp to the big dict which saves EVERY task. Open new slice, set the new current task to task_index and save current_start to timestamp. After loop - close final slice if one is open. Pass slices to matplotlib and draw.
- OK, so this wasn't needed lol - I don't have itm or swo
- I added UART traces using the same packets and structure.
- Now, we start with priority inversion and its testing. We are basically testing our Mutex Lock and Unlock systems. We have tested priority starvation, running the LEDs at different priorities and speeds using OS_Sleep(), and the basic bootup sequence. Now, after mutex, we can test something for the semaphores as well then the malloc then we're done.

- Even though more technical notes are available on the detailed doc, we will add some stuff here for our understanding. To test a semaphore, we must make a producer and a consumer task. The Consumer task must have the higher priority. It is the one that calls `OS_Wait()` and runs an LED with a Loop that burns CPU cycles. Producer calls the `OS_Signal()` and turns on a status LED.
- Consumer Task: Process Data, read data out of the buffer, must wait if the buffer is empty, and WAKES UP THE producers.
- Producer Task: Generate data, write data into the buffer, must wait if the buffer is full, wake up consumer after adding data.
- SOO, semaphore producer-consumer system is used for synchronization and signaling.
- When the system boots, the high-priority consumer runs first. It hits `OS_Wait()` and sees that the counter is 0 so BLOCKS itself
- The Producer runs and sleeps. After 500ms, it wakes up and calls `OS_Signal()`. The kernel increments the counter, sees that the consumer has been waiting, and marks it as READY. Since it is a higher priority, there is a context switch and Consumer starts running. It loops back and hits `OS_Wait()` to put itself back to sleep. The CPU is handed back to the Producer exactly where it left off. The producer toggles its LED and goes back to sleep.
- In reality, Consumer LED turns a fraction of a second before Producer, but we can't see that with bare eyes, so it seems like they are blinking together.
- Let's test the malloc feature. Test 1: Basic alloc and free. Allocate a block, write a value into it, free it, allocate again and verify you get usable memory back. This proves the basic path work.

New Syntax Learned (Python):

- So our .log file is binary. Not text, so we can't do line-by-line reads. We need to do 4 bytes at a time in binary mode. Check python script for more clarification.
- So, I also made a mistake on the extraction part - x is a bytes object, not an integer. Therefore, I couldn't use bit shift directly. Use `struct.unpack` to convert 4 bytes into a `uint32_t` so we can use it ->

import struct -> value in uint32 = struct.unpack(">I", 4-byte value)[0]. SOO, in this case, '>' is a format specifier for big-endian byte order (most significant byte at lowest memory address). 'I' is format code for unsigned int. '[0]' just allows you to take the actual value you need, as the function returns a tuple and we only need the first part.

New Syntax learned (C):

- Function Pointers in C -> return_type (*pointer_name) (parameter_types);
- There is something called CMSIS tools to have assembly stuff in C code. First used example was for the CONTROL Register. We can use __get_CONTROL() and __set_CONTROL(...)
- __set_PSP(...) - load a task's stack pointer into the process stack pointer (PSP)
- __set_PSP(...) - Enables switching from the Main Stack Pointer (MSP) to a dedicated stack for user applications commonly used in RTOS to isolate task stacks. Takes a top of stack uint32_t.
- __disable_irq() disabled all interrupts
- When working with pointer arithmetic, PLEASE typecast DURING the arithmetic statement and don't move them around every other line. Example: Current = (Header*)((uint8_t*)(Current + 1) + Current->size);
- If we want a new line in the macro (mostly while writing functions), use the \ (backslash). Make sure there are no characters after the \ in that line.

New Syntax learned (Assembly):

- EXPORT: directive to let Keil know this symbol exists and is a function.
- THUMB: Thumb Instruction (16bit)
- AREA: declares a named code section
- CPSID I -> instruction that disables interrupts by setting the PRIMASK special register to 1. the I stands for IRQ (Interrupt Request)
- CPSIE I -> enable interrupts

- MRS (Move to Register from Special Register) instruction in ARM assembly transfers the contents of a PSR such as the Current Program Status Register (CPSR) into a general purpose register.
- STMDB: Store Multiple Decrement Before -> Stores the contents of multiple registers into memory. It decrements the base register (usually the SP) *before* each data transfer.
- ! Is the **writeback suffix** - it tells the processor to update the base register with the new address calculated during the instruction.
- STMDB R0!, {R4-R11} -> stores registers R4 through R11 into memory starting at the address in R0. CRUCIAL: Updates R0 to the new final address, essentially moving the pointer so it's ready for the next operation.
- AAPCS Rules: Before we hide the LR value, we must know where to put it. The AAPCS divides the CPU's general-purpose registers into two strict categories:
 - a. Caller-Saved (scratch) Registers: R0,R1,R2,R3,R12,LR. When the OS_Schedule() runs, the C code is allowed to completely destroy whatever data is sitting in these registers to do its math. If you care about the data in these registers, you must save them before the BL instruction.
 - b. Callee-Saved Registers: R4-R11. The C compiler is bound to preserve these. If the scheduler needs to use R4 for some math, it will push R4 to the stack when the function starts, and pop it back to its original value right before it returns.
- LDmia: Load Multiple, Increment After. Pops R4-R11 (in my case) off the next task's stack - uses ! For writeback.

Lessons from mistakes:

- Use (void) if function doesn't take anything (don't just do ()).
- All registers are 32 bits, so use uint32_t pointers
- Make sure you know the difference of grabbing an address (using &) and just grabbing a simple value.
- So if we are trying to look at a piece of

- RAM and want to go from top of stack to down, we need to pre-decrement. So decrement then assign something. But a better way in my case was to subtract the psp at stackTop pointer by 16. This way, the arm will count upwards, towards a ceiling where there is content in between. We get a proper top to bottom stack pop.
- When trying to point something to a function pointer, ensure that they have the same type. In my case, there was an uint32_t integer trying to point to a void type function. Ensure that you type cast in certain situations.
- Handler Mode always uses the MSP.
- For main.c, my planning and steps were wrong. Remember that both the SysTick and PendSV handler are interrupt handlers and are never called from main.c. - Correct order before system is handed off to the OS -> PLL_init -> GPIO Init -> OS Init (Zeros TCBs, nulls CurrentTask) -> Create Tasks (populate the fake frames) -> SysTick init -> Start OS (no return from here gng)
- Something I have been struggling with: Include Guards (or header guards). No file gets let into the compilation party twice. Examples seen below
- #ifndef TASKS_H (If not defined): like a check
- #define TASKS_H: basically the heading/stamp
- #endif: The end of the guarded section
- Indents in assembly
- Error: type name does not allow storage class to be specified -> I was trying to declare a typedef inside a struct scope. Need to specify it OUTSIDE the struct, then add in the struct.
- To do priority scheduling correctly, we have to separate the searching from the assigning - something I did together in one loop.
- Ok so make sure that even if you are assigning a variable a certain value later, after initialising without a value, you are doing it in a definitive way. I did it in an if statement. This means that if that if statement never ran, my variable would have garbage data.
- I got stuck on the semaphore.c implementation of OS_Signal(). I was unsure as to how to REMOVE the task from the waiting_tasks[] array. I initially thought FIFO, with the assumption that Mutex handles Priority Inheritance. However, even though the Mutex does that, we NEED the semaphore to filter based on priorities

which makes it cleaner and safer

- In terms of C code, I made mistakes while writing this semaphore.c implementation. I had a global instance of a semaphore pointer, and I was initialising it as well - this will not compile - Only declarations are allowed at file scope. The right approach is to MAKE a function that initialises the semaphore struct smp. This is because C does not allow lines like `smp->counter = 0` OUTSIDE a function.
- Also, because we need the functions for various semaphores, we need to set `Semaphore* smp` as a parameter to both functions - they both can't run on a global semaphore pointer instance.
- I forgot about the generic, untyped memory address syntax - it is `void*`. We used this for `os_malloc`, as we need to hide the actual
- Make sure when using `sizeof()`, you are differentiating between a pointer input and an actual data input like a struct - on my arm cortex, `sizeof(pointer)` will always be 4 bytes.
- Keep an eye out for pointer math problems. Like when I was allocating the heap array, I made it `uint32_t` initially which meant that the `heap_array[1024]` allocated 1024 words in silicon which meant we allocated 4096 byte heap.
- You cannot use `i++` to walk a heap.
- So I created a circular dependancy while adding the idle task. In order to access it in `os_core.c`, I included `tasks.h` but that file already needed `os_core.c` to function. Therefore, the solution was to use `extern` at the top of `os_core.c`. declare the `idleTask` function using the `extern` keyword, and don't include anything and we're good.
- Some mistakes I made were in pointer math - One more Rule: Let's say we have two pointers, both TCB struct type. They are 32 bytes of RAM. Let's say the address difference between the two is `0x40` = 64 bytes. However, the compiler will do $64/32 = 2$. This means that the current task is at index 2.
- OK, so even though we made sure not to make mistakes regarding assuming hardware capabilities, I got carried away for the tracing part and wrote a python file for the SWO log file. Turns out, the CMSIS-DAP debugger I have does not support SWO. SOOO, we are going to use UART.
- Ok so during my testing phase of malloc, I was trying to see if

different mallocs overwrites one another in terms of addresses. So, my sense of overwriting was wrong. We must check pointers, and not straight values

- To prove your allocator works, you have to prove that the block of RAM given to arr2 starts *after* the block of RAM given to arr1. Since arr1 requested 5 uint32_t integers (which is 20 bytes), the physical memory address of arr2 must be strictly greater than arr1 + 5 (which in pointer math means "arr1 plus 5 integer spaces").

Hurdles in the Project: (LESSONS LEARNED)

- The hurdles started coming in as soon as the first build/translate button was hit. Most of the issues will come during testing - Any other Coding related and architecture related issues and lessons learned are above.

1. HardFault Failure:

Symptom: The OS crashed into HardFault (Figured using the CMSIS-DAP Debugger) the exact millisecond Start_OS() was called to boot at first

Root Cause: I attempted to trigger the context switcher using a Supervisor Call (SVC) after manually flipping the CONTROL register to use the PSP. Because the hardware exceptions immediately push 8 registers to the active stack, the CPU violently pushed the garbage state of the Start_OS function directly onto the newly forged PSP stack frame, corrupting the Initial PC.

The Fix: Removed the CONTROL Register manipulation. By executing SVC #0 as is, while still on the MSP, the CPU safely saved the garbage state to the MSP. Inside the SVC_Handler, loading the value 0xFFFFFDD into LR (EXC_RETURN) and executing BX LR forced the CPU to dynamically switch to the PSP and pop the uncorrupted stack frame.

2. Pointer math vs integer Math (stack alignment bug):

Symptom: CPU attempted to pop the fake stack frame but pulled the wrong registers, causing a crash.

Root Cause: Pointer arithmetic automatically scales by the size of the data type. When attempting to manually adjust the PSP by 8 slots to bypass the software registers (R4-R11), the code originally cast the pointer to an integer first: `(uint32_t)CurrentTask->psp + 8`. This added exactly 8 bytes of data rather than the requires 32 bytes.

The Fix: `(uint32_t)(CurrentTask->psp + 8)`. This is word-aligned pointer math, moving the pointer exactly 8 words down.

3. Kernel Architecture Log: OS_Sleep & Priority Scheduling Verification Defect Resolution: The "Everybody Asleep" Trap

- Symptom: Following the implementation of `OS_Sleep()`, the kernel exhibited unpredictable behavior across multiple physical board resets. LEDs would either blink erratically at incorrect frequencies or enter a "stuck ON" state, indicating a high-speed PWM crash.
- Root Cause: A critical flaw was discovered in the `OS_Schedule` context-switching logic. When all active application tasks called `OS_Sleep()`, the system's READY list became entirely empty. Because the scheduler's array-search loop failed to find a valid READY task, it did not update the `indexTracker`. Consequently, the OS blindly forced the last sleeping task back into the RUNNING state, completely ignoring its sleep timer and causing rapid, infinite preemption loops.
- The Fix: Implemented a system Idle Task to guarantee the READY list is never completely empty.
 - The Idle Task was hardcoded directly into the `Init_OS` boot sequence and placed at the final index of the `task_array`.
 - It consists of an infinite `while(1)` loop and is assigned the absolute lowest priority in the system (Priority 0).
 - New Architectural Guideline: All user application tasks must be

created with Priority 1 or higher. Priority 0 is now strictly reserved for the OS to prevent starvation of the Idle Task.

System Verification: Preemption & Round-Robin Time Slicing

To prove the mathematical stability of the priority scheduler, a mixed-priority test was conducted with four distinct tasks.

Test Configuration:

- Task 1: Priority 1 | OS_Sleep(500)
- Task 2: Priority 1 | OS_Sleep(500)
- Task 3: Priority 1 | No Sleep (Infinite while(1) loop *after* single LED toggle)
- Task 4: Priority 3 (Highest) | OS_Sleep(250)

Observed Results & Architectural Proof:

1. Preemption Confirmed: LED4 blinked perfectly at 250ms. Because Task 4 held the highest priority, the scheduler successfully and violently preempted Tasks 1, 2, and 3 precisely when Task 4's sleep timer expired.
2. Round-Robin Confirmed: LED1 and LED2 blinked synchronously at 500ms. Because they shared Priority 1, the scheduler correctly identified the tie and time-sliced the CPU between them via modulo arithmetic while Task 4 slept.
3. Task Trapping Confirmed: LED3 turned ON once and remained solidly ON. Because Task 3's toggle instruction lived outside its infinite loop, and it never called OS_Sleep(), it never voluntarily yielded the CPU. The OS correctly gave it CPU time, but the task simply spun in an empty void, proving that tasks without sleep commands will aggressively consume available cycles without altering hardware states.

System Verification: Intentional Priority Starvation

To further validate the ruthless nature of strict priority scheduling, an intentional starvation test was executed.

Test Configuration:

- Task 3: Set to Highest Priority | Implemented with a busy-wait infinite loop (no OS_Sleep).
- Task 1, 2, 4: Set to Lower Priorities.

Observed Results & Architectural Proof:

- LED1 and LED3 remained permanently ON, while LEDs 2 and 4

never activated.

- The State Machine Reality: Task 1 booted first, toggled its LED ON, and was immediately preempted by the first SysTick hardware interrupt. The scheduler evaluated the array, identified Task 3 as the highest priority, and handed it the CPU. Because Task 3 utilized a busy-wait loop instead of yielding, it permanently locked the CPU at 100% utilization. Task 1 was left permanently frozen in the exact state it was preempted (LED ON), perfectly demonstrating system starvation.

4. Verification: Priority Inversion

Theory: High priority task is blocked by a low priority task, causing delays. This happens because a lower priority task holds a mutex which it can not release until it is done. However, while it is running, a higher priority task runs which then finishes, but does not hand back the CPU to the low priority task.

Therefore, it does not release the mutex, which means the highest priority task can not use that mutex and therefore not run.

We need to set up a test for this.

Let's Say task 1 is my intern. It has the lowest priority of 1. Task 2 is priority 2 and preempts task 1 at some time. Task 3 is priority 3 and needs the mutex task 1 has. Task 4 just keeps running, showing that the system is beating. This means we run task 1 first, run task 2 after a certain delay, and run task 3 after both the previous ones start running. We need task 3 to access the mutex while it is locked. If priority inversion takes place, Task 3 should never be able to run, while task 1 is stuck on its first iteration, and task 2 keeps running.

However, we accounted for priority inversion and therefore, priority inheritance should take place where task 1 gets task 3's priority, runs, unlocks the mutex and then task3 runs. Let's code this now.

Corrected expected observations

Your theory description is good but the observations need adjusting:

Without priority inheritance (broken kernel):

- task1 locks mutex, sleeps 500ms inside it
- task2 wakes at 300ms, priority 2, starts hogging CPU
- task3 wakes at 600ms, priority 3, tries to lock mutex — gets BLOCKED
- task1 needs to wake up and unlock but task2 keeps preempting it since task1 is only priority 1
- task3 is stuck forever, task1 never gets CPU, task2 hogs everything
- LED3 never blinks, LED1 stops after first toggle, LED2 stays on

With priority inheritance (your kernel):

- task1 locks mutex, sleeps 500ms inside it
- task2 wakes at 300ms, starts hogging
- task3 wakes at 600ms, tries mutex, gets blocked — task1's priority elevates to 3
- task1 now outranks task2, gets CPU, wakes from sleep, unlocks mutex
- task3 immediately gets mutex, LED3 starts blinking
- task1 priority drops back to 1, task2 resumes hogging
- LED1 blinks once per cycle, LED3 blinks, LED4 blinks steadily, LED2 stays on

Rest of the stuff on the google doc

Sources:

<https://interrupt.memfault.com/blog/cortex-m-rtos-context-switching> - Theory

<https://github.com/freertos> - This will be used to verify my approach. I will compare my implementation with this and explain what I do differently or same compared to theirs.

<https://angom.myweb.cs.uwindsor.ca/teaching/cs330/WritingOS.pdf>

Some Code I need to save for my Tests: (only remembered to start saving after priority inversion test) - anything before that, I can recreate using my notes

Priority Inversion Test: How I set up the tasks.

```

#include "tasks.h"
#include "tm4c1294ncpdt.h"
#include "SysTick.h"
void task1(void) { //Blinks LED1
    while (1) {
        Mutex_Lock(&Mx);
        GPIO_PORTN_DATA_R ^= 0x02; //Toggle Second Bit
for LED1
        for (volatile uint32_t i = 0; i
< 3000000; i++) {}
        Mutex_Unlock(&Mx);
        OS_Sleep(100);
    }
}
void task2(void) { //Blinks LED2
    OS_Sleep(500);
    while (1) {
        GPIO_PORTN_DATA_R ^= 0x01; //Toggle First Bit
for LED2
        for (volatile uint32_t i = 0; i
< 1000000; i++) {}
    }
}
void task3(void) { //Blinks LED3
    OS_Sleep(1000);
    while (1) {
        Mutex_Lock(&Mx); //shouldnt be able to access
        GPIO_PORTF_DATA_R ^= 0x10; //Toggle Fourth Bit
for LED3
        for (volatile uint32_t i = 0; i < 500000; i++)
        {}
        Mutex_Unlock(&Mx);
        OS_Sleep(500);
    }
}
void task4(void) { //Blinks LED4
    while (1) {
        GPIO_PORTF_DATA_R ^= 0x01; //Toggle First Bit
for LED4
        OS_Sleep(250);
    }
}

```

```

}
void idleTask(void) {
    while (1) { //Runs infinitely
    }
}

```

Code for Semaphore Producer Consumer Test:

```

#include "tasks.h"
#include "tm4c1294ncpdt.h"
#include "SysTick.h"
void ConsumerTask (void) {
    while (1) {
        OS_Wait(&smp); //Sets the counter to 0 if it is
at 1. If it is at 0, block the current task and put it
on the waiting_tasks array
        GPIO_PORTN_DATA_R ^= 0x02;
        OS_Sleep(100);
    }
}

void ProducerTask(void) {
    while (1) {
        OS_Sleep(500); //Sleep for some time to simulate
waiting for a message
        OS_Signal(&smp); //Calls the OS_Signal function
on the semaphore which sets the counter to 1 and works
on waiting tasks
        GPIO_PORTN_DATA_R ^= 0x01; //Turn ON status LED
        OS_Sleep(100);
    }
}

void task3(void) { //Blinks LED3
    while (1) {
        //GPIO_PORTF_DATA_R ^= 0x10; //Toggle Fourth Bit
for LED3
        OS_Sleep(500);
    }
}

void task4(void) { //Blinks LED4

```

```

        while (1) {
            //GPIO_PORTF_DATA_R ^= 0x01; //Toggle First Bit
for LED4
            OS_Sleep(500);
        }
    }
void idleTask(void) {

    while (1) { //Runs infinitely

    }

}

```

Detailed notes in the doc

Testing malloc and free:

```

#include "tasks.h"
#include "tm4c1294ncpdt.h"
#include "SysTick.h"
#include "os_malloc.h"
void MallocTest(void) {

    uint32_t* arr = (uint32_t*)os_malloc(5 *
sizeof(uint32_t)); //Allocating memory for 5 uint32_t
integers
    if (arr == NULL) { OS_Fault(); }
    arr[0] = 4;
    if (arr[0] == 4) { GPIO_PORTN_DATA_R ^= 0x01; }
//led 2

    os_free(arr);

    uint32_t* array = (uint32_t*)os_malloc(5 *
sizeof(uint32_t));

    array[0] = 67;
    if (array[0] == 67) { GPIO_PORTF_DATA_R ^= 0x01; }
//LED 4

    while (1) { OS_Sleep(1000); }
}

```

```

}
void task2(void) { //Blinks LED2
    while (1) {
        //GPIO_PORTN_DATA_R ^= 0x01; //Toggle First Bit
for LED2
        OS_Sleep(500);
    }
}
void task3(void) { //Blinks LED3
    while (1) {
        //GPIO_PORTF_DATA_R ^= 0x10; //Toggle Fourth Bit
for LED3
        OS_Sleep(500);
    }
}
void task4(void) { //Blinks LED4 (Used as a system
heartbeat in this malloc test)
    while (1) {
        //GPIO_PORTF_DATA_R ^= 0x01; //Toggle First Bit
for LED4
        OS_Sleep(500);
    }
}
void idleTask(void) {

    while (1) { //Runs infinitely

    }

}

```

As a result, the OS was able to allocate memory and be written to and be read from. The LEDs lit up as expected and did not throw an OS_Fault(). Therefore, the basic malloc and free both work.

Test 2 for malloc: we are allocating multiple places, filling them up, and checking for overlap

```

#include "tasks.h"
#include "tm4c1294ncpdt.h"
#include "SysTick.h"
#include "os_malloc.h"
void MallocTest(void) {

    //allocating 20 bytes of memory
    uint32_t* arr1 = (uint32_t*)os_malloc(5 *
sizeof(uint32_t)); //Allocating memory for 5 uint32_t
integers
    if (arr1 == NULL) {
        OS_Fault(); //If the allocation fails, we fault
the system.
    }
    for (uint32_t i = 0; i < 5; i++) {
        arr1[i] = i + 1; //Filling the allocated memory
with values from 1 to 5
    }
    //allocating 4 bytes of memory
    uint32_t* arr2 = (uint32_t*)os_malloc(sizeof(uint32
_t)); //Allocating memory for 1 uint32_t integer
    if (arr2 == NULL) {
        OS_Fault(); //If the allocation fails, we fault
the system.
    }
    arr2[0] = 10;
    //allocating 8 bytes of memory
    uint32_t* arr3 = (uint32_t*)os_malloc(2 *
sizeof(uint32_t)); //Allocating memory for 2 uint32_t
integers
    if (arr3 == NULL) {
        OS_Fault(); //If the allocation fails, we fault
the system.
    }
    arr3[0] = 15;
    arr3[1] = 20;

    //now we need to test for overlap
    if (arr2 < (arr1 + 5)) { OS_Fault(); } //We know
that arr2 must start AFTER arr1 + 5 integer spaces - if
it is less than the end of arr2's block, they overlap
and it's faulty

```

`if (arr3 < (arr2 + 1)) { OS_Fault(); }` //We know that arr3 must start AFTER arr2's one integer space - if it is less than the end of arr2's block, they overlap and it's faulty

```
//Check if the values are still the same for arr1
for (uint32_t i = 0; i < 5; i++) {
    if (arr1[i] != (i + 1)) {
        OS_Fault();
    }
}

while (1) { OS_Sleep(1000); }

}
void task2(void) { //Blinks LED2
    while (1) {
        //GPIO_PORTN_DATA_R ^= 0x01; //Toggle First Bit
for LED2
        OS_Sleep(500);
    }
}
void task3(void) { //Blinks LED3
    while (1) {
        //GPIO_PORTF_DATA_R ^= 0x10; //Toggle Fourth Bit
for LED3
        OS_Sleep(500);
    }
}
void task4(void) { //Blinks LED4 (Used as a system
heartbeat in this malloc test)
    while (1) {
        //GPIO_PORTF_DATA_R ^= 0x01; //Toggle First Bit
for LED4
        OS_Sleep(500);
    }
}
void idleTask(void) {

    while (1) { //Runs infinitely
```



```

    }

}

```

TEST 3:

```

#include "tasks.h"
#include "tm4c1294ncpdt.h"
#include "SysTick.h"
#include "os_malloc.h"
void MallocTest(void) {

    uint32_t count = 0;

    while (1) {

        uint32_t* arr = (uint32_t*)os_malloc(50);
        if (arr == NULL) { break; }
        count++;

    }

    for (volatile uint32_t i = 0; i < count; i++)
    { GPIO_PORTF_DATA_R ^= 0x01; OS_Sleep(500); }

    while (1) { OS_Sleep(1000); }

}

void task2(void) { //Blinks LED2
    while (1) {
        //GPIO_PORTN_DATA_R ^= 0x01; //Toggle First Bit
for LED2
        OS_Sleep(500);
    }
}

void task3(void) { //Blinks LED3
    while (1) {
        //GPIO_PORTF_DATA_R ^= 0x10; //Toggle Fourth Bit
for LED3
        OS_Sleep(500);
    }
}

```

```

    }
}
void task4(void) { //Blinks LED4 (Used as a system
heartbeat in this malloc test)
    while (1) {
        //GPIO_PORTF_DATA_R ^= 0x01; //Toggle First Bit
for LED4
        OS_Sleep(500);
    }
}
void idleTask(void) {

    while (1) { //Runs infinitely

    }

}

```

OMG IT WORKSSSSSSSS